

Lesson 7: Structures

Before discussing classes, this lesson will be an introduction to data structures similar to classes. Structures are a way of storing many different variables of different types under the same name. This makes it a more modular program, which is easier to modify because its design makes things more compact. It is also useful for databases.

The format for declaring a structure (in C++, it is different in C) is

```
struct Tag {  
    Members  
};
```

Where Tag is the name of the entire type of structure. To actually create a single structure the syntax is

```
Tag name_of_single_structure;
```

To access a variable of the structure it goes

```
name_of_single_structure.name_of_variable;
```

For example:

```
struct example {  
    int x;  
};  
example an_example; //Treating it like a normal variable type  
an_example.x = 33; //How to access it's members
```

Here is an example program:

```
struct database {  
    int id_number;  
    int age;  
    float salary;  
};  
  
int main()  
{  
    database employee; //There is now an employee variable that has modifiable  
                        // variables inside it.  
    employee.age = 22;  
    employee.id_number = 1;  
    employee.salary = 12000.21;  
}
```

The struct database declares that database has three variables in it, age, id_number, and salary. You can use database like a variable type like int. You can create an employee with the database type as I did above. Then, to modify it you call everything with the 'employee.' in front of it. You can also return

structures from functions by defining their return type as a structure type. For instance:

```
database fn();
```

I will talk only a little bit about unions as well. Unions are like structures except that all the variables share the same memory. When a union is declared the compiler allocates enough memory for the largest data-type in the union. Its like a giant storage chest where you can store one large item, or a small item, but never the both at the same time.

The '.' operator is used to access different variables inside a union also.

As a final note, if you wish to have a pointer to a structure, to actually access the information stored inside the structure that is pointed to, you use the -> operator in place of the . operator. All points about pointers still apply.

A quick example:

```
#include <iostream>

using namespace std;

struct xampl {
    int x;
};

int main()
{
    xampl structure;
    xampl *ptr;

    structure.x = 12;
    ptr = &structure; // Yes, you need the & when dealing with structures
                      // and using pointers to them
    cout<< ptr->x;    // The -> acts somewhat like the * when used with pointers
                      // It says, get whatever is at that memory address
                      // Not "get what that memory address is"
    cin.get();
}
```